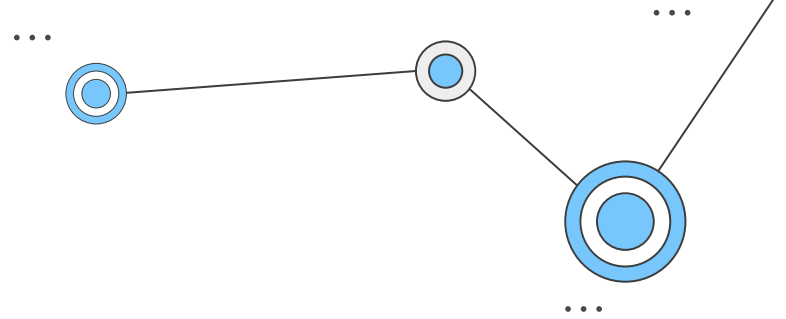
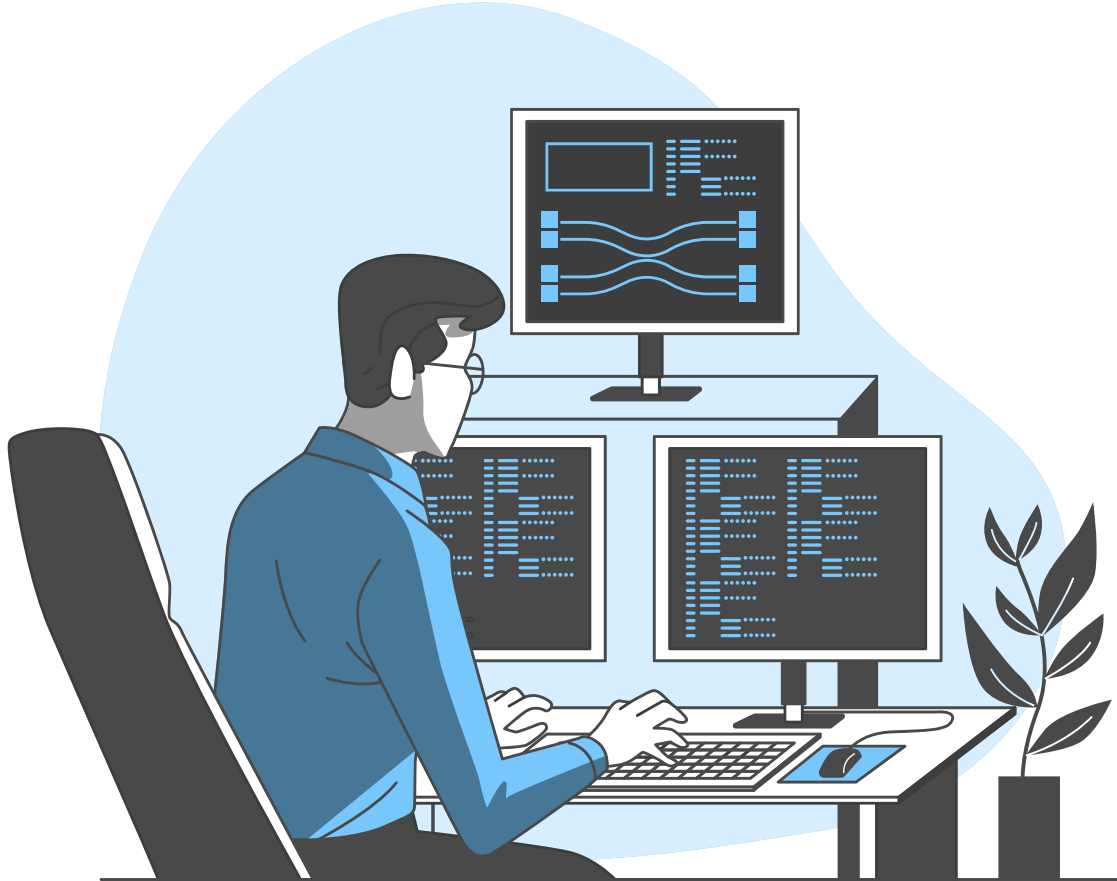


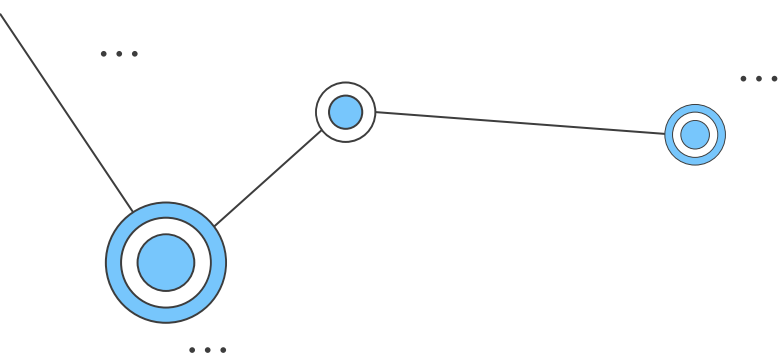


BASIS DATA LANJUT

Pertemuan 02

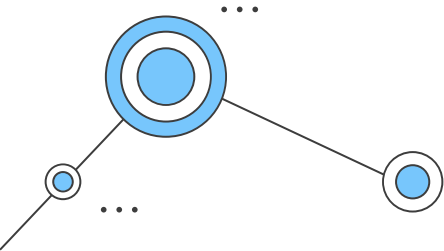
Query Dasar (DDL dan DML)





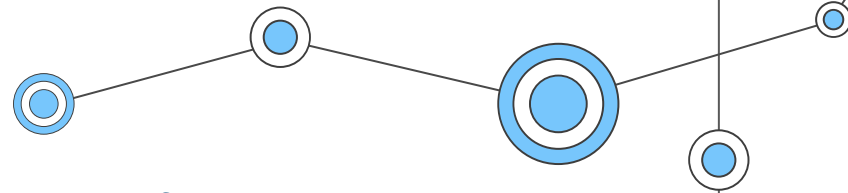
OUTLINE

- Definisi DDL dan DML
- Sintaks DDL dan DML
- Tipe Data Khas PostgreSQL
- Constraint





Data Definition Language (DDL)



DDL adalah bagian dari SQL yang digunakan untuk **mendefinisikan struktur dan objek dalam basis data**, seperti membuat, mengubah, atau menghapus tabel, indeks, dan skema.

DDL tidak berhubungan langsung dengan manipulasi data, tetapi lebih fokus pada pengelolaan objek database itu sendiri.

Terdapat 3 perintah/klausa utama yang sering digunakan pada DDL

Klausa	Deskripsi	Contoh
CREATE	Perintah untuk membuat objek	CREATE DATABASE, CREATE TABLE, CREATE INDEX, CREATE VIEW
ALTER	Perintah untuk melakukan perubahan struktur/objek, seperti tabel atau kolom	ALTER TABLE
DROP	Perintah untuk menghapus objek, seperti database, table, view, index	DROP DATABASE, DROP TABLE, DROP VIEW, DROP INDEX





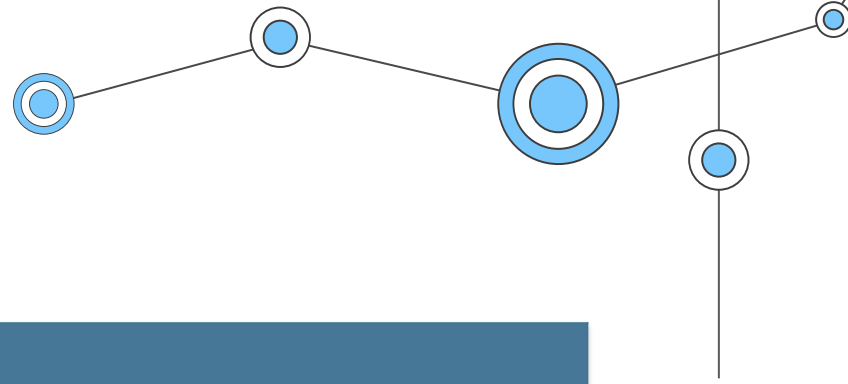
DDL – CREATE

- Membuat database baru:

```
CREATE DATABASE namadatabase;
```

- Contoh:

```
CREATE DATABASE kampusdb;
```



DDL – CREATE

- Membuat tabel baru:

```
CREATE TABLE namatabel(  
    namakolom1 tipedata constraint,  
    namakolom2 tipedata,  
    ... .  
);
```

- Contoh:

```
CREATE TABLE mahasiswa(  
    nim SERIAL,  
    nama VARCHAR(30) NOT NULL,  
    angkatan INT,  
    email VARCHAR(50)  
);
```

DDL – ALTER

- Menambahkan primary key

```
ALTER TABLE namatabel ADD CONSTRAINT namakolom_pk PRIMARY  
KEY (namakolom) ;
```

Contoh:

```
ALTER TABLE mahasiswa ADD CONSTRAINT mahasiswa_pk PRIMARY KEY(nim);
```

- Menambahkan kolom

```
ALTER TABLE namatabel ADD COLUMN namakolom tipedata;
```

Contoh:

```
ALTER TABLE mahasiswa ADD COLUMN alamat TEXT;
```

DDL – ALTER (1)

- Menghapus kolom

```
ALTER TABLE namatabel DROP COLUMN namakolom tipedata;
```

Contoh:

```
ALTER TABLE mahasiswa DROP COLUMN email;
```

- Mengubah tipe data

```
ALTER TABLE namatabel ALTER COLUMN namakolom TYPE  
tipedatabaru;
```

Contoh:

```
ALTER TABLE mahasiswa ALTER COLUMN alamat VARCHAR(100);
```

DDL – ALTER (2)

- Mengubah nama kolom

```
ALTER TABLE namatabel RENAME COLUMN namakolom tipedata;
```

Contoh:

```
ALTER TABLE mahasiswa RENAME COLUMN email;
```



DDL – DROP

- Menghapus tabel

```
DROP TABLE namatabel;
```

- DROP TABLE CASCADE

Memungkinkan PostgreSQL ikut menghapus semua obyek yang bergantung pada tabel tersebut, misalnya FOREIGN KEY di tabel lain yang mereferensikan tabel tersebut, VIEW/Materialized view yang memakai tabel tersebut

```
DROP TABLE namatabel CASCADE;
```

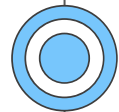
- Menghapus database

```
DROP DATABASE namadatabase;
```

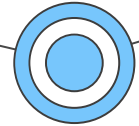


DDL PostgreSQL vs DDL MySQL

- Secara sintaks query DDL pada PostgreSQL sama dengan MySQL.
- Yang membedakan adalah tipe datanya (lebih detail akan dijelaskan pada slide berikutnya)
 - MySQL → menggunakan AUTO_INCREMENT
 - PostgreSQL → tidak ada AUTO_INCREMENT, tetapi SERIAL / BIGSERIAL
- MySQL tidak memiliki DROP TABLE CASCADE. Jika tabel direferensikan oleh FK, FK harus dihapus terlebih dahulu, atau DROP tabel anaknya dulu.



Data Manipulation Language (DML)



DML adalah bagian dari SQL yang digunakan untuk **memanipulasi data** dalam tabel database.

DML digunakan untuk melakukan operasi CRUD (Create, Retrieve/Read, Update, Delete) pada data di dalam database.

Terdapat beberapa klausa utama yang sering digunakan pada DML

Klausa	Deskripsi
INSERT	Perintah untuk menambahkan baris data baru (create) pada table
SELECT	Perintah untuk mengambil baris data dari table (retrieve/read), SELECT juga bisa masuk ke ranah DRL
UPDATE	Perintah untuk mengubah baris data dari table (update)
DELETE	Perintah untuk menghapus baris data dari table (delete)
TRUNCATE	Perintah untuk menghapus semua baris dari table (mengosongkan isi table)



DML – INSERT

- Menambahkan satu data ke tabel (single row insert):

```
INSERT INTO namatabel (namakolom1, namakolom2, ...)  
VALUES ('isidata1', isidata2, ...);
```

Contoh:

```
INSERT INTO mahasiswa (nama, angkatan, email)  
VALUES ('Adi', 2023, 'adi@gmail.com');
```

- Menambahkan satu data ke tabel (single row insert):

```
INSERT INTO namatabel (namakolom1, namakolom2, ...)  
VALUES ('isidata1', isidata2, ...),  
      ('isidata1', isidata2, ...);
```



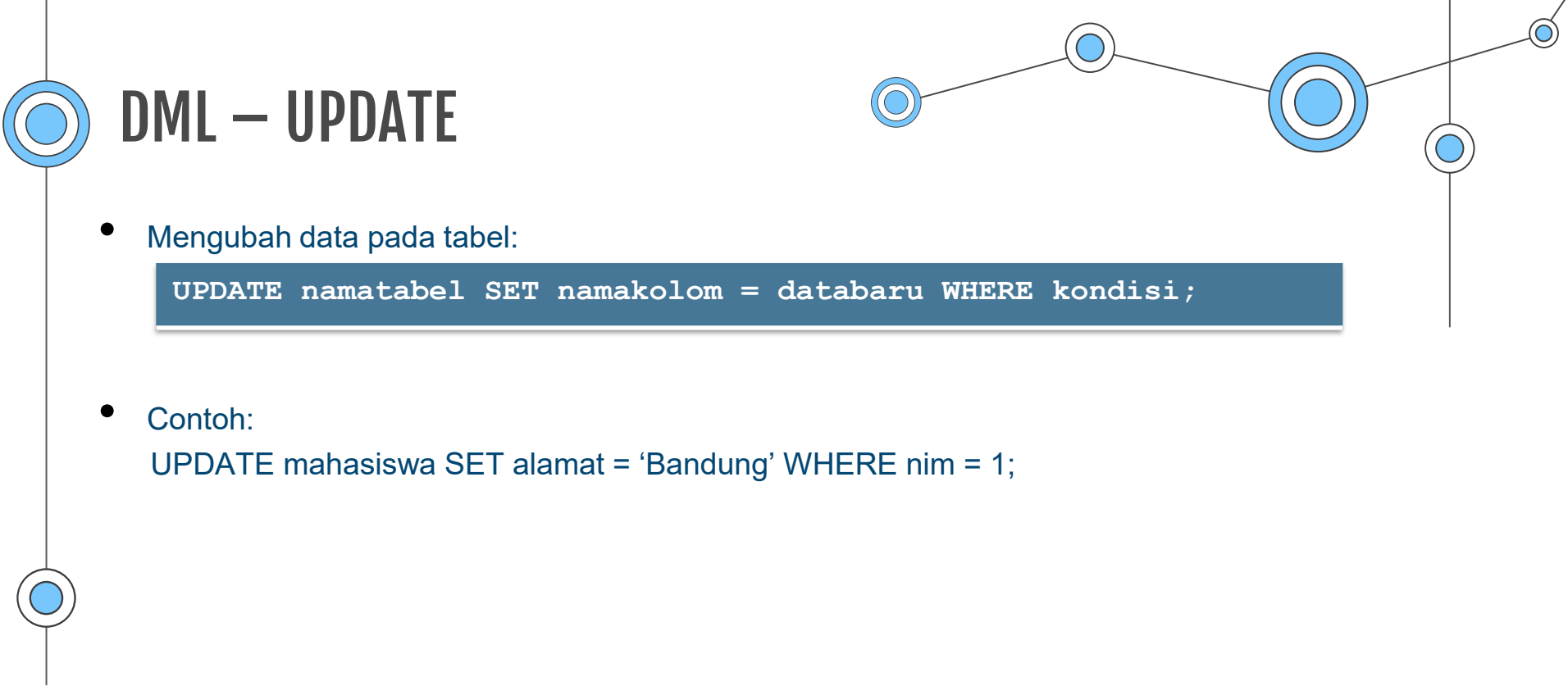
DML – SELECT

- Menampilkan semua data di satu tabel:

```
SELECT * FROM namatabel;
```

- Menampilkan data tertentu di satu tabel:

```
SELECT * FROM namatabel WHERE kondisi;
```



DML – UPDATE

- Mengubah data pada tabel:

```
UPDATE namatabel SET namakolom = databaru WHERE kondisi;
```

- Contoh:

```
UPDATE mahasiswa SET alamat = 'Bandung' WHERE nim = 1;
```



DML – DELETE

- Menghapus data pada tabel:

```
DELETE FROM namatabel WHERE kondisi;
```

Contoh:

```
DELETE FROM mahasiswa WHERE nim = 1;
```

- Menghapus semua baris pada tabel (tidak disarankan):

```
DELETE FROM namatabel
```

DML – TRUNCATE

- Mengosongkan semua baris dengan sangat cepat

```
TRUNCATE TABLE namatabel1, namatabel2, ... opsitruncate
```

- Dapat menambahkan beberapa opsi truncate:
 - ☐ RESTART IDENTITY: reset sequence yang dimiliki tabel ke awal
 - ☐ CONTINUE IDENTITY (default): tidak mereset sequence
 - ☐ CASCADE: ikut truncating tabel lain yang mereferensikan tabel via FK
 - ☐ RESTRICT: mencegah jika ada tabel lain yang mereferensikan

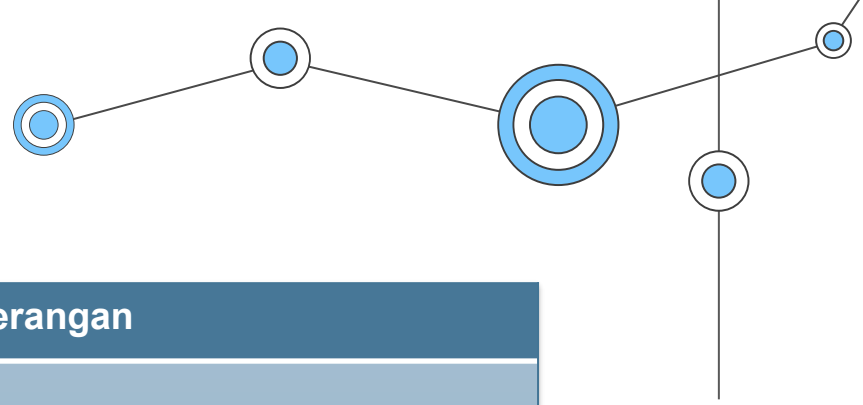
DML PostgreSQL vs DML MySQL

- Secara sintaks query DML PostgreSQL sama dengan MySQL
- Perbedaannya ada pada TRUNCATE, perbandingannya:

Aspek	MySQL	PostgreSQL
Reset auto-increment	Default/otomatis	Melalui opsi RESTART IDENTITY
Opsi CASCADE ke tabel lain	Tidak ada	Ada TRUNCATE ... CASCADE
Rollback	Tidak bisa	Bisa (dalam transaksi)
Trigger row-level	Tidak memicu	Bisa (perubahan setiap baris)



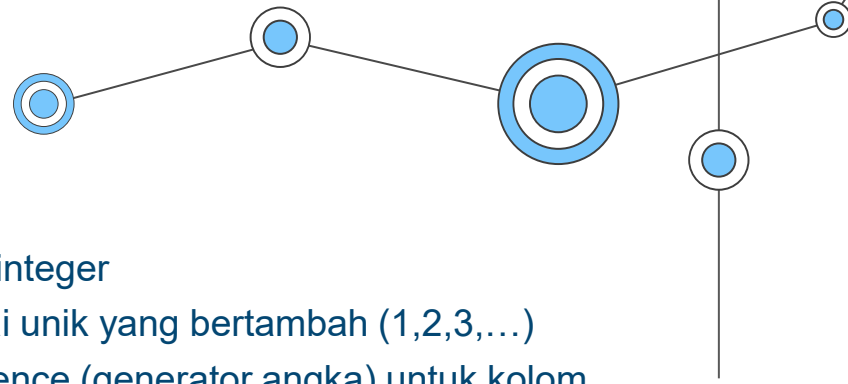
Tipe Data Khas PostgreSQL



Tipe Data	Keterangan
SERIAL	Auto increment integer
UUID	Universally Unique Identifier
JSONB	JavaScript Object Notation, tetapi dalam format biner terkompresi
ARRAY	Satu kolom dapat menyimpan banyak nilai sekaligus
MONEY	Menyimpan nilai mata uang
RANGE	Menyimpan rentang nilai



Tipe Data: SERIAL



- Tipe data khusus PostgreSQL untuk auto increment integer
- Setiap kali insert, PostgreSQL otomatis memberi nilai unik yang bertambah (1,2,3,...)
- Di belakang layar, PostgreSQL akan membuat sequence (generator angka) untuk kolom bertipe SERIAL
- Variasi tipe data SERIAL:

Variasi	Rentang nilai
SMALLSERIAL	1 – 32.767
SERIAL	1 – 2.147.438.674 (INT 32-bit)
BIGSERIAL	1 – 9.223.372.036.854.775.807 (INT 64-bit)

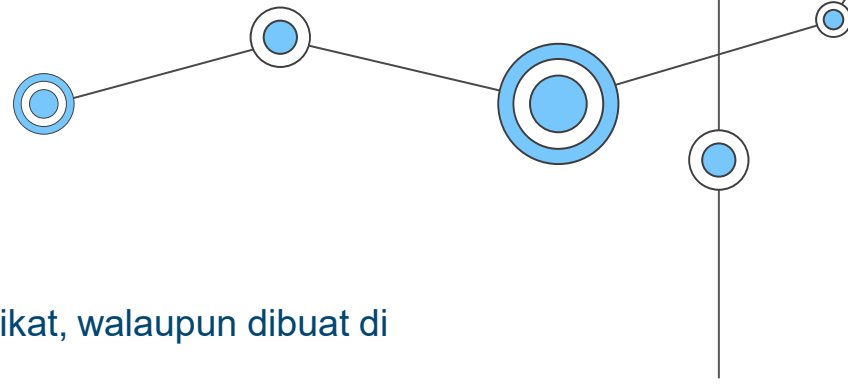




Tipe Data: UUID

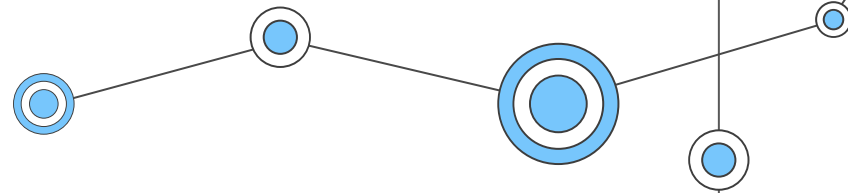
- Universally Unique Identifier
- Nilai berupa string 128-bit yang hampir mustahil duplikat, walaupun dibuat di komputer/server berbeda
- Format berupa 36 karakter (32 hex + 4 tanda strip)
- Contoh UUID:

550e8400-e29b-41d4-a716-446655440000





Tipe Data: JSONB



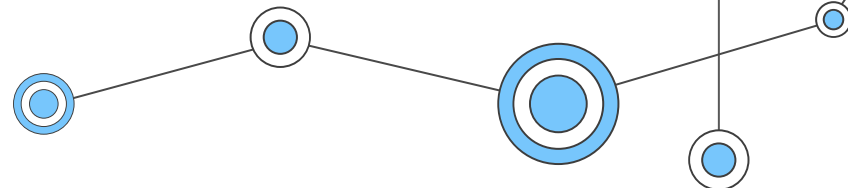
- Tipe data PostgreSQL untuk menyimpan data JSON (JavaScript Object Notation), tetapi dalam format biner terkompresi
- Dibandingkan dengan tipe JSON biasa, JSONB lebih cepat untuk query dan indexing
- Perbandingan JSON vs JSONB

Aspek	JSON	JSONB
Penyimpanan	Menyimpan teks mentah JSON	Menyimpan biner terstruktur
Kecepatan	Lebih lambat	Lebih cepat, bisa pakai indeks
Urutan Key	Dipertahankan	Tidak dipertahankan
Duplikasi Key	Bisa disimpan	Dihilangkan





Tipe Data: ARRAY



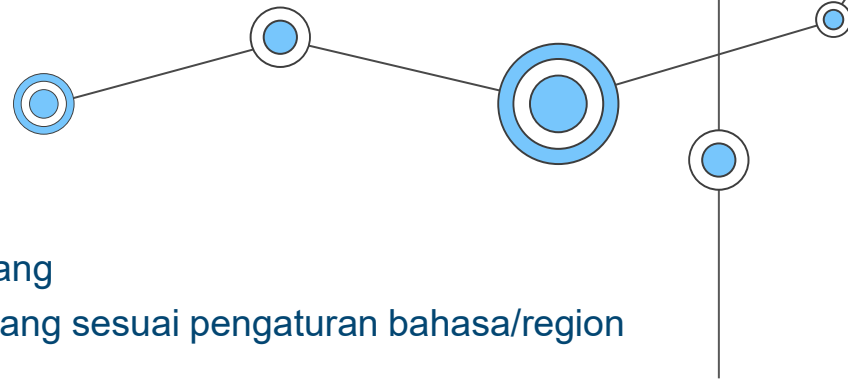
- Tipe data PostgreSQL yang memungkinkan satu kolom menyimpan banyak nilai sekaligus, mirip seperti array dalam bahasa pemrograman
- Contoh: kolom 'keahlian' bisa menyimpan ['AI', 'Data Mining', 'Basis Data'] dalam satu baris.
- Array digunakan untuk data yang memang berupa list singkat, tetapi jika data akan sering dianalisis (join/filter kompleks) sebaiknya tetap dibuat tabel relasi M:N, bukan array





Tipe Data: MONEY

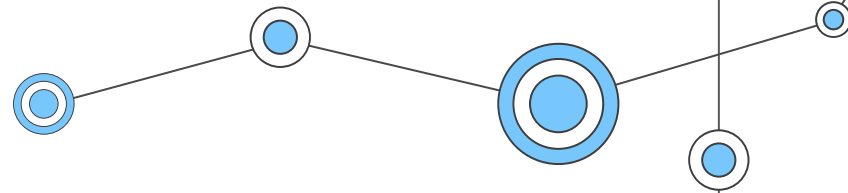
- Tipe data PostgreSQL untuk menyimpan nilai mata uang
- Nilainya otomatis ditampilkan dengan symbol mata uang sesuai pengaturan bahasa/region server PostgreSQL
- Dapat dilakukan perhitungan (SUM, +, -), tetapi untuk operasi yang lebih kompleks kurang fleksibel, lebih baik dikonversi ke dalam format NUMERIC





Tipe Data: RANGE

- Tipe data PostgreSQL untuk menyimpan rentang nilai
- Bisa digunakan untuk angka, tanggal, waktu
- PostgreSQL menyediakan operator khusus untuk mengecek apakah suatu nilai masuk dalam range
- Format dasar:
 - [] inklusif (termasuk batas),
 - () eksklusif (tidak termasuk batas)
- Contoh:
 - [1, 5] → 1 sampai 5, termasuk 1 dan 5
 - (1, 5) → 1 sampai 5, termasuk 1, tidak termasuk 5



- Tipe range built-in:

Tipe	Isi rentang
Int4range	Rentang INTEGER
Int8range	Rentang BIGINT
Numrange	Rentang NUMERIC
Daterange	Rentang DATE
Tsrange	Rentang TIMESTAMP tanpa zona
Tstzrange	Rentang TIMESTAMP dengan zona

CONSTRAINT

Constraint	Keterangan
PRIMARY KEY	Unik dan NOT NULL
UNIQUE	Nilai kolom/kolom gabungan tidka boleh duplikat
FOREIGN KEY	Keterkaitan antar tabel
CHECK	Validasi logika (rentang nilai, relasi antar kolom, dll)
NOT NULL	Kolom wajib berisi nilai
EXCLUDE (khas PostgreSQL)	Hanya dimiliki PostgreSQL, melarang kombinasi yang “bertabrakan” menurut operator tertentu (memerlukan indeks GIST/SP-GIST). Misal: jadwal: satu ruang tidak boleh punya rentang waktu yang beririsan

Kesimpulan MySQL vs PostgreSQL

Aspek	MySQL	PostgreSQL
AUTO-INCREMENT	AUTO_INCREMENT	Tipe data SERIAL
DROP TABLE ... CASCADE	✗	✓
ON DELETE/ UPDATE CASCADE	✓	✓
TRUNCATE ... CASCADE	✗	✓
Constrain EXCLUDE	✗	✓

Thanks!

Do you have any questions?



Team Teaching Matakuliah Basis Data Lanjut
JTI POLINEMA